Guion de la presentación del sistema de reservas por seis grupos

Guion y fuentes de treinta diapositivas

Un bloque por grupo y una demostración conjunta del sistema de reservas; las notas mantienen la procedencia y límites de cada afirmación.

Índice

Actualizar el índice en Word con F9.

DOC-06-02 · Treinta diapositivas · 20 de octubre de 2026

La presentación conjunta sigue los seis grupos del documento rector y culmina con una demostración funcional. Las notas explican el caso propio y separan evidencia local de hipótesis pendientes. La guía DOC-07-02 desarrolla la preparación y el registro de la demostración.

Diapositiva 1 Sistema de reservas

Abrir con el sistema construido por el seminario. El caso propio es la reserva de cupos, no una empresa externa ni una aplicación de venta. Distinguir el éxito funcional local de las hipótesis de escalabilidad todavía pendientes.

Fuentes: documentacion/fase-01/fuentes/organizacion_grupos_seminario.docx; documentacion/fase-01/fuentes/caso_exito_kubernetes_openshift.docx

Diapositiva 2 La pregunta del seminario

La pregunta conecta los seis equipos. No basta con mostrar que la pantalla responde: se debe verificar inventario, una confirmación por intención y calidad de servicio. El rango de millones procede del documento rector; no fue ejecutado en el laboratorio actual.

Fuentes: documentacion/fase-01/fuentes/caso_exito_kubernetes_openshift.docx

Diapositiva 3 Seis grupos sobre un mismo sistema

Respetar la distribución del documento original: 4+4+4+4+3+3=22. Cada representante coordina semanalmente con G6. La investigación y la exposición siguen este mismo orden.

Fuentes: documentacion/fase-01/fuentes/organizacion_grupos_seminario.docx

Diapositiva 4 El recorrido de una reserva

Mostrar el recorrido funcional existente. El frontend ofrece catálogo, disponibilidad, reserva, consulta y salida. El comprobante aparece después de la confirmación de la base, no después de encolar una solicitud.

Fuentes: frontend/src/app/App.tsx; aplicacion/reservas/store.py

Diapositiva 5 Arquitectura del sistema construido

Responsable: Grupo 1.

G1 explica fronteras y responsabilidades. El frontend compilado sirve assets y /api mediante proxy; la API aplica autorización y negocio. Diferenciar la aplicación con worker de la topología de medición S19 sin worker.

Fuentes: frontend/container/nginx.conf; aplicacion/reservas/store.py; evidencias/frontend-s19/topology.json

Diapositiva 6 Objetos de Kubernetes aplicados a reservas

Responsable: Grupo 1.

Relacionar cada objeto con un componente del diagrama. Explicar que etcd conserva estado del clúster, no las reservas del negocio. Los manifiestos preparados requieren admisión y verificación real.

Fuentes: https://kubernetes.io/docs/concepts/overview/; frontend/deploy/base/frontend.yaml

Diapositiva 7 Arquitectura actual y arquitectura propuesta

Responsable: Grupo 1.

La hipótesis original compara microservicios y monolito; no hay todavía un comparador equivalente ejecutado. G1 entrega arquitectura base y ficha reproducible antes de atribuir ventajas al estilo arquitectónico.

Fuentes: documentacion/fase-01/fuentes/organizacion_grupos_seminario.docx; evidencias/frontend-s18/runtime.json

Diapositiva 8 Qué se escala en el sistema de reservas

Responsable: Grupo 2.

Aplicar HPA y KEDA a cargas diferentes para no crear controladores que compitan por el mismo Deployment. No presentar un dibujo de réplicas como evidencia de escalado ejecutado.

Fuentes: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/; https://keda.sh/docs/2.21/scalers/apache-kafka/

Diapositiva 9 Recursos de Pods y capacidad de nodos

Responsable: Grupo 2.

Separar dimensión vertical, horizontal y de nodos. Activarlas juntas impediría atribuir un efecto. G2 conserva el estudio de todos los mecanismos y ordena su evaluación.

Fuentes: https://kubernetes.io/docs/concepts/workloads/autoscaling/vertical-pod-autoscale/; https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

Diapositiva 10 Comparación para evaluar el autoescalado

Responsable: Grupo 2.

Declarar presupuesto máximo de recursos de cada condición para distinguir el efecto del control de asignar más capacidad. Registrar también estabilización y solicitudes pendientes. No existe todavía un contraste local que pruebe la hipótesis de G2.

Fuentes: documentacion/fase-01/fuentes/DOC-01-02-investigacion-seis-grupos.md

Diapositiva 11 Resiliencia de una operación de reserva

Responsable: Grupo 3.

Un frontend saludable no certifica salud de PostgreSQL. PDB no evita fallos físicos ni toda eliminación de Pods. Una sola base sigue siendo un punto de fallo.

Fuentes: https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-probes/; https://kubernetes.io/docs/concepts/workloads/pods/disruptions/

Diapositiva 12 Seguridad y Service Mesh

Responsable: Grupo 3.

La prueba local de UID existe en S18. El grupo debe obtener SCC efectiva y eventos del clúster. Service Mesh puede agregar control y telemetría pero no sustituye una transacción ni garantiza efectos externos únicos.

Fuentes: evidencias/frontend-s18/runtime.json; https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/authentication_and_authorization/managing-pod-security-policies; https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/

Diapositiva 13 Disponibilidad con una definición medible

Responsable: Grupo 3.

G3 entrega matriz de fallos y protocolo. S18 verificó caída y recuperación de una API exclusiva en Compose, no tolerancia de un nodo OpenShift. El porcentaje debe interpretarse solo en la ventana y método del experimento.

Fuentes: evidencias/frontend-s18/runtime.json; documentacion/fase-01/fuentes/organizacion_grupos_seminario.docx

Diapositiva 14 Una confirmación conserva tres registros coherentes

Responsable: Grupo 4.

Las cajas describen la unidad lógica de la transacción, no una instrucción para ejecutar commits separados. Consultar disponibilidad antes no protege frente a dos clientes concurrentes. La escritura condicional y restricciones de base deciden la reserva.

Fuentes: aplicacion/reservas/store.py; https://www.postgresql.org/docs/current/transaction-iso.html

Diapositiva 15 Idempotencia y consistencia

Responsable: Grupo 4.

Explicar la diferencia entre clave privada de autenticación, clave de idempotencia e identificador de comprobante. No mostrar claves reales en proyección. CAP trata particiones y garantías distribuidas, no una elección genérica de dos letras para cualquier sistema.

Fuentes: aplicacion/reservas/store.py; https://doi.org/10.1145/564585.564601

Diapositiva 16 Kafka como ampliación del sistema de reservas

Responsable: Grupo 4.

La publicación se propone después del commit. Responder solo porque la solicitud se encoló requeriría otro contrato, por ejemplo estado pendiente. G4 entrega esquema, topic, particionado, retención y deduplicación; KEDA estudiaría el consumidor.

Fuentes: https://kafka.apache.org/documentation/#design; documentacion/fase-01/fuentes/caso_exito_kubernetes_openshift.docx

Diapositiva 17 Métricas de plataforma y de negocio

Responsable: Grupo 5.

G5 necesita relacionar las señales con run_id y tiempo. No usar claves ni UUID por reserva como etiquetas de métricas. Los rechazos válidos y los fallos técnicos se reportan por separado.

Fuentes: https://opentelemetry.io/docs/concepts/signals/; aplicacion/reservas/api.py

Diapositiva 18 Instrumentación actual y tablero propuesto

Responsable: Grupo 5.

No presentar un dashboard diseñado como si estuviera instalado. La precisión del 95 por ciento exige TP FP FN, ventanas etiquetadas y demora; no equivale a confianza estadística ni garantiza un punto exacto de degradación.

Fuentes: https://prometheus.io/docs/practices/histograms/; https://opentelemetry.io/docs/concepts/signals/; evidencias/frontend-s19/summary.json

Diapositiva 19 Lo que S19 midió en nuestro sistema

Responsable: Grupo 5.

Valores recalculados desde JSONL: 18.514697, 19.016653 y 19.461934 ms, con 120 observaciones de creación por corrida. Hubo 360 reservas nuevas medidas y veinte de calentamiento. Se registró limitación de CPU en API y generador y ningún evento OOM en los intervalos. No se midieron HPA, Kafka, clúster ni HTTPS.

Fuentes: evidencias/frontend-s19/summary.json; evidencias/frontend-s19/database-audit.json

Diapositiva 20 Qué significa un millón de transacciones

Responsable: Grupo 6.

La operación de negocio no equivale a una descarga de JavaScript o una lectura de catálogo. S19 generó 360 reservas nuevas durante las corridas medidas. El símbolo distinto de significa que no se deben intercambiar las unidades.

Fuentes: evidencias/frontend-s19/summary.json; documentacion/fase-01/fuentes/DOC-01-02-investigacion-seis-grupos.md

Diapositiva 21 Protocolo para contrastar la hipótesis general

Responsable: Grupo 6.

Los niveles suman 26M. Multiplicar por dos condiciones y tres repeticiones da 156M, sin reenvíos ni lecturas. Esa escala exige aprobar recursos y almacenamiento; no es una promesa para la laptop. Los umbrales de latencia y disponibilidad del documento son propuestas por congelar.

Fuentes: documentacion/fase-01/fuentes/caso_exito_kubernetes_openshift.docx; documentacion/fase-01/fuentes/DOC-01-02-investigacion-seis-grupos.md

Diapositiva 22 Conclusiones con evidencia e incertidumbre

Responsable: Grupo 6.

Tres repeticiones son un mínimo exploratorio. No promediar p95 para llamarlo p95 conjunto. No concluir todo el rango desde un resultado favorable en 10M. Hoy H0 y H1 a escala OpenShift siguen sin contraste suficiente.

Fuentes: https://www.itl.nist.gov/div898/handbook/prc/section3/prc311.htm; https://itl.nist.gov/div898/handbook/prc/section2/prc241.htm

Diapositiva 23 Demostración conjunta del sistema de reservas

Antes de la exposición comprobar salud, acceso y cupos de un entorno exclusivo. Puede utilizarse S18 si continúa disponible y se registra el inventario real. No usar la base medida S19 para nuevas reservas de demostración porque alteraría su auditoría.

Fuentes: documentacion/fase-07/fuentes/DOC-07-02-guia-seis-grupos.md

Diapositiva 24 Acceso catálogo y disponibilidad

G3 prepara el acceso por canal privado y oculta la proyección durante la entrada. G1 muestra componentes mientras G4 registra el valor inicial A. Si no hay evento o cupo, detener la reserva y usar evidencia preparada; no forzar datos del experimento.

Fuentes: frontend/src/session/LaboratoryAccess.tsx; frontend/src/availability/AvailabilityPage.tsx

Diapositiva 25 Confirmación y consulta de una reserva

G4 comprueba cantidad e identificador; G5 anota operación y resultado; G6 registra la verificación. No hay cancelación ni pago que demostrar. La reserva de demo consume un cupo real de su base exclusiva.

Fuentes: frontend/src/reservation/ReservationReceipt.tsx; frontend/src/session/ExitSession.tsx

Diapositiva 26 Reenvíos y recuperación sin duplicación

El frontend bloquea el doble envío. Para demostrar el contrato se usan las pruebas aisladas S18/S19 o un ensayo preparado y auditado. La recuperación controlada debe ensayarse antes; no detener servicios de otros sprints frente al público.

Fuentes: evidencias/frontend-s18/deployment-real.json; evidencias/frontend-s19/database-audit.json; documentacion/fase-10/sprints/S10/fuentes/DOC-S10-01-entrega.md

Diapositiva 27 Logros actuales y metas por investigar

Esta es la separación que debe conservar toda conclusión. El éxito local es real y útil; no necesita atribuirse capacidades de plataforma no ejecutadas. No hay worker en S19 ni evidencia de entrega externa exactamente una vez.

Fuentes: evidencias/frontend-s18/revision-documental.json; evidencias/frontend-s19/revision-documental.json; documentacion/fase-01/fuentes/organizacion_grupos_seminario.docx

Diapositiva 28 Integración del trabajo de los grupos

Los equipos trabajan de forma coordinada y no como seis exposiciones inconexas. G6 integra a todos; no puede ejecutar la hipótesis general antes de recibir arquitectura, políticas, datos e instrumentos verificables.

Fuentes: documentacion/fase-01/fuentes/organizacion_grupos_seminario.docx

Diapositiva 29 Qué entrega cada grupo

La evaluación debe valorar comprensión y trazabilidad, no solo una herramienta instalada. Las referencias técnicas son Kubernetes, Red Hat, PostgreSQL, Kafka, KEDA, OpenTelemetry, Prometheus y NIST, además de los dos documentos rectores y evidencias del proyecto. Consultar la bibliografía completa en DOC-01-02.

Fuentes: documentacion/fase-01/fuentes/DOC-01-02-investigacion-seis-grupos.md; documentacion/fase-07/fuentes/DOC-07-02-guia-seis-grupos.md

Diapositiva 30 Conclusión del caso de éxito propio

Cerrar mostrando cómo cada grupo contribuyó al problema inicial. La investigación no promete confirmar H1; debe poder refutarla o declarar insuficiencia de evidencia. El sistema propio es el

caso central y las metas de millones, HPA, Kafka y disponibilidad permanecen explícitas como investigación pendiente.

Fuentes: documentacion/fase-01/fuentes/DOC-01-02-investigacion-seis-grupos.md